KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 2 __str__과 __repr__

print했을 때
붕어빵이 보이게 만들기

```
class Bread:
    def __str__(self):
        return "팥 붕어빵 1,000원"

print(a)
# 팥 붕어빵 1,000원
# (전에는 <__main__.Bread object at ...>)
```

오늘의 순서

1 왜 필요한가

print했을 때 낯선 글자, 파이썬이 찾아 부르는 메서드, 가장 짧은 __str__

4–6쪽

2 __str__

붕어빵마다 다른 글자, 불리는 곳, intro와 비교

7–9쪽

3 __repr__

남는 낯선 글자, '팥'과 팥, 주문표 모양, 한 장 표, 하나만 만든다면

10–15쪽

4 붕어빵 여러 개 다루기

for, 컴프리헨션, key 정렬, 재고 바꾸기, if로 막기

16–20쪽

5 오류와 함정

print를 쓴 __str__, 숫자 돌려주기, 밑줄 하나, 글자 더하기

21–24쪽

6 실습

노점의 하루 두 장, 카페 메뉴판

25–27쪽

3쪽은 노트북 준비, 28–29쪽은 막힐 때와 한 장 요약이에요.

들어가기

24에서 이어서 해요

- ✓ class, __init__, self — 24
- ✓ 메서드와 a.intro() — 24
- ✓ f-string과 칸 맞추기 — 13
- ✓ return과 print의 차이 — 19
- ✓ sorted와 key=lambda — 22

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

1. 왜 필요한가

숫자와 리스트는 잘 보이는데, 붕어빵은 안 보여요

[1]
```python
print(3)
print("팥")
print([1, 2])
```

```
3
팥
[1, 2]
```

파이썬은 우리 붕어빵을 몰라요

3, "팥", 리스트는 파이썬이 만든 틀이라 보여 주는 법을 알아요. Bread는 우리가 만든 틀이라 몰라요.

[2]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
print([a, b])
```

```
<__main__.Bread object at 0x7f6735bb38c0>
[<__main__.Bread object at 0x7f6735bb38c0>, <__main__.Bread object at 0x7f6735dc4a50>]
```

1. 왜 필요한가

정해진 순간에 파이썬이 틀에서 찾아 부르는 메서드

틀 class Bread

__init__	찍을 때
__str__	print할 때
__repr__	리스트나 셀에서 볼 때
intro	우리가 부를 때

Bread("팥", 1000) → __init__

print(a) → __str__

셀 마지막 줄의 a → __repr__

print([a, b]) → __repr__

앞뒤 밑줄 두 개 = 파이썬과 약속한 이름

a.__str__()처럼 직접 부르지 않아요. 이름만 맞게 적어 두면 그 순간에 파이썬이 찾아서 불러요. 없으면 기본 모양(<... object at 0x...>)을 써요.

1. 왜 필요한가

가장 짧은 __str__: 글자 하나를 돌려주기

[3]

```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):                    # 새로 넣은 두 줄
        return "붕어빵이에요"

a = Bread("팥", 1000)
print(a)
```

붕어빵이에요

print(a)가 바뀌었어요

낯선 글자 대신 __str__이 돌려준 글자가 나왔어요.

return이에요

print가 대신 찍을 수 있게 글자를 돌려줘요. 안에서 print하지 않아요.

print(a)를 치면 파이썬이 a의 __str__을 불러 그 글자를 찍어요.

2. __str__

__str__에 self의 값을 넣어 붕어빵마다 다르게

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):                                  메뉴판 글씨
        return f"{self.flavor} 붕어빵 {self.price:,}원"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
print(b)
```

Python

```
팥 붕어빵 1,000원
슈크림 붕어빵 1,200원
```

a — Bread 붕어빵

flavor	"팥"
price	1000

팥 붕어빵 1,000원

손님에게 보여 주는 글씨

__str__은 메뉴판처럼 사람이 읽기 좋은 글자를 돌려줘요.

2. __str__

print 말고도 글자가 필요한 곳에서 불려요

[2]
```python
print(str(a))
print(f"주문: {a}")
print("오늘의 빵:", b)
```

```
팥 붕어빵 1,000원
주문: 팥 붕어빵 1,000원
오늘의 빵: 슈크림 붕어빵 1,200원
```

[3]
```python
msg = "포장할게요: " + str(a)
print(msg)
```

```
포장할게요: 팥 붕어빵 1,000원
```

`str(a)` 12의 타입 바꾸기. 글자로 바꿔요

`f"{a}"` f-string 중괄호 안

`print("...", b)` 쉼표 뒤에 넣어도

글자로 바꿀 때마다

파이썬이 붕어빵을 글자로 바꿔야 할 때 __str__을 불러요.

2. __str__

24의 intro()와 무엇이 다른가요

intro (24)

```
    def intro(self):
        print(self.flavor, "붕어빵")

a.intro()
print(a)   # 여전히 <... object>
```

__str__

```
    def __str__(self):
        return f"{self.flavor} 붕어빵"

print(a)
f"주문: {a}"
```

	intro	__str__
누가 부르나요	우리가 `a.intro()`로	print, str, f-string이 알아서
무엇을 하나요	안에서 바로 print	글자를 return
다른 글자와 잇기	안 돼요	f-string, 더하기로 이어요

3. __repr__

__str__을 만들었는데도 낯선 글자가 남아요

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a
```
print 없이 Python

```
<__main__.Bread at 0x7febaa53c6e0>
```

셀 마지막 줄

print 없이 이름만 적으면 __str__을 쓰지 않아요.

리스트 안

리스트를 print해도 안의 붕어빵은 __str__을 쓰지 않아요.

[2]
```python
print([a, b])
```
Python

```
[<__main__.Bread object at 0x7febaa53c6e0>, <__main__.Bread object at 0x7febaa7fb890>]
```

3. __repr__

이 차이는 "팥"에서 이미 봤어요

[3]
```python
"팥"
```

```
'팥'
```

[4]
```python
print("팥")
print(["팥", "슈크림"])
print(str("팥"))
print(repr("팥"))
```

```
팥
['팥', '슈크림']
팥
'팥'
```

str 손님용

팥

print, str()

repr 만든 사람용

'팥'

셀 마지막 줄, 리스트 안, repr()

따옴표가 있고 없고

글자라는 걸 알 수 있게 따옴표까지 보여 주는 게 repr이에요. 리스트 안의 글자에 따옴표가 붙던 이유예요.

3. __repr__

__repr__까지 만들면 어디서나 보여요

[5]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

    def __repr__(self):                                    주문표 글씨
        return f"Bread('{self.flavor}', {self.price})"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a
```

Python

```
Bread('팥', 1000)
```

[6]
```python
print(a)
print([a, b])
```

Python

```
팥 붕어빵 1,000원
[Bread('팥', 1000), Bread('슈크림', 1200)]
```

코드처럼 적는 게 약속이에요

Bread('팥', 1000)을 복사해서 셀에 붙여 넣으면 같은 붕어빵이 다시 찍혀요. 만든 사람이 무엇이 들었는지 한눈에 봐요.

3. __repr__

언제 어느 쪽이 불리는지 한 장으로

이렇게 쓰면	불리는 메서드	둘 다 있을 때	아무것도 없을 때
`Bread("팥", 1000)`	`__init__`	찍기	재료 없이 찍기
`print(a), str(a), f"{a}"`	`__str__`	팥 붕어빵 1,000원	`<... object at 0x...>`
셀 마지막 줄의 a	`__repr__`	Bread('팥', 1000)	`<... at 0x...>`
`print([a, b])`	안의 값은 __repr__	`[Bread('팥', 1000), ...]`	`[<... object ...>, ...]`
`a.intro()`	우리가 부른 것	우리가 부를 때만	AttributeError

print는 손님 글씨(__str__), 그 밖에 개발자가 볼 때는 주문표 글씨(__repr__)예요.

3. __repr__

하나만 만든다면 __repr__이에요

[7]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __repr__(self):                                    __repr__만
        return f"Bread('{self.flavor}', {self.price})"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
print([a, b])
```

Python

```
Bread('팥', 1000)
[Bread('팥', 1000), Bread('슈크림', 1200)]
```

print도 __repr__을 써요

__str__이 없으면 print는 __repr__을 빌려 써요. 반대로는 빌려 쓰지 않아요.

언제 둘 다 만드나요

손님용 글씨가 따로 필요할 때 __str__을 더해요. 영수증, 메뉴판처럼요.

3. __repr__

참고, 읽기만: len과 +도 같은 원리예요

[8]
```python
print(len([1, 2, 3]))
print([1, 2, 3].__len__())
print(3 + 4)
print((3).__add__(4))
```

```
3
3
7
7
```

따라 치지 않아도 돼요

리스트 틀에는 __len__이, 숫자 틀에는 __add__가 들어 있어요. 이 과정에서는 __init__, __str__, __repr__ 세 개만 만들어요.

`len(xs)` → 파이썬이 찾는 메서드 `__len__`

`a + b` → 파이썬이 찾는 메서드 `__add__`

4. 붕어빵 여러 개 다루기

붕어빵 여러 개를 리스트에 담아 하나씩 찍어요

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

    def __repr__(self):
        return f"Bread('{self.flavor}', {self.price})"

breads = [Bread("팥", 1000), Bread("슈크림", 1200), Bread("피자", 1500)]
for b in breads:
    print(b)
```

```
팥 붕어빵 1,000원
슈크림 붕어빵 1,200원
피자 붕어빵 1,500원
```

for로 꺼낸 b

b는 붕어빵 하나라서 print(b)는 __str__을 써요.

4. 붕어빵 여러 개 다루기

23의 컴프리헨션으로 맛만, 가격만, 비싼 것만

[2]
```python
print([b.flavor for b in breads])
print(sum([b.price for b in breads]))
print([b for b in breads if b.price >= 1200])
```

Python

```
['팥', '슈크림', '피자']
3700
[Bread('슈크림', 1200), Bread('피자', 1500)]
```

`b.flavor` 붕어빵마다 맛 칸만 꺼내 모아요

`b.price` 가격만 모아 sum에 넘겨요

`if b.price >= 1200` 붕어빵을 통째로 골라 담아요. 리스트라서 __repr__이 보여요

__repr__이 없었다면 마지막 줄은 <... object at 0x...>뿐이었어요.

4. 붕어빵 여러 개 다루기

22의 key=lambda로 가격순 정렬과 가장 비싼 붕어빵

[3]
```python
print(sorted(breads, key=lambda b: b.price, reverse=True))
print(max(breads, key=lambda b: b.price))
```

```
[Bread('피자', 1500), Bread('슈크림', 1200), Bread('팥', 1000)]
피자 붕어빵 1,500원
```

[4]
```python
print(sorted(breads))
```

```
TypeError: '<' not supported between instances of 'Bread' and 'Bread'
```

key가 꼭 필요해요

붕어빵끼리는 무엇으로 크기를 비교할지 몰라요. key로 가격을 비교하라고 알려줘요.

max는 붕어빵 하나

max가 돌려준 건 리스트가 아니라 붕어빵 하나라서 print가 __str__을 써요.

4. 붕어빵 여러 개 다루기

메서드로 재고와 판매 개수를 바꿔요

[5]

```
class Stall:
    def __init__(self, name, stock):
        self.name = name
        self.stock = stock
        self.sold = 0

    def sell(self, qty):                                    값 바꾸기
        self.stock = self.stock - qty
        self.sold = self.sold + qty

    def __str__(self):
        return f"{self.name}: 남은 {self.stock}개, 판매 {self.sold}개"

s = Stall("팥", 10)
print(s)
s.sell(3)
print(s)
```

Python

```
팥: 남은 10개, 판매 0개
팥: 남은 7개, 판매 3개
```

일	stock	sold
찍기	10	0
sell(3)	7	3

상태가 남아요

sell을 부를 때마다 self의 칸이 바뀌고, 바뀐 값은 다음에 부를 때까지 남아요. print(s)는 지금 상태를 보여 줘요.

4. 붕어빵 여러 개 다루기

재고가 모자라면 팔지 않게 if로 막아요

[6]

```python
class Stall:
    def __init__(self, name, stock):
        self.name = name
        self.stock = stock
        self.sold = 0

    def sell(self, qty):
        if qty > self.stock:
            return "재고가 모자라요"
        self.stock = self.stock - qty
        self.sold = self.sold + qty
        return "판매 완료"

s = Stall("팥", 5)
print(s.sell(3))
print(s.sell(4))
print(s.stock, s.sold)
```

17의 if

Python

```
판매 완료
재고가 모자라요
2 3
```

return에서 바로 끝나요

19에서 배운 대로 return을 만나면 아래 줄은 실행하지 않아요.

값이 지켜져요

4개를 팔려고 했지만 재고가 2개라 그대로예요.

5. 오류와 함정

__str__ 안에서 print하면 returned non-string

[1]

```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        print(self.flavor, "붕어빵")

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
```

return이 없어요

Python

```
팥 붕어빵
TypeError: __str__ returned non-string (type NoneType)
```

읽는 법

__str__이 글자 대신 None을 돌려줬어요. return이 없으면 None이에요(19).

고치기

print 대신 return f"{self.flavor} 붕어빵"

5. 오류와 함정

숫자를 돌려줘도 returned non-string

[2]

```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return self.price                    숫자

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
```

Python

```
TypeError: __str__ returned non-string (type int)
```

읽는 법

글자(str)가 아니라 숫자(int)를 돌려줬어요.

고치기

return str(self.price) 또는 return f"{self.price}원"

5. 오류와 함정

밑줄이 하나면 오류 없이 그냥 무시돼요

[3]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def _str_(self):                                    _str_
        return f"{self.flavor} 붕어빵"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
```
Python

```
<__main__.Bread object at 0x7fc8a2a6bcb0>
```

오류가 안 나서 더 찾기 어려워요

파이썬은 약속된 이름 __str__만 찾아요. _str_은 평범한 메서드예요.

고치기

앞뒤로 밑줄 두 개씩 __str__, __repr__

5. 오류와 함정

글자에 붕어빵을 바로 더하면 can only concatenate

[4]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print("주문: " + a)
```
글자 + 붕어빵 Python

```
TypeError: can only concatenate str (not "Bread") to str
```

읽는 법

글자에는 글자만 더할 수 있어요. 더하기는 __str__을 부르지 않아요.

고치기

"주문: " + str(a), f"주문: {a}", print("주문:", a)

6. 실습

붕어빵 노점의 하루 ①: 노점 틀 만들기

[1]

```python
class Stall:
    def __init__(self, flavor, price, stock):
        self.flavor = flavor
        self.price = price
        self.stock = stock
        self.sold = 0

    def sell(self, qty):
        if qty > self.stock:
            return f"{self.flavor} 재고가 모자라요"
        self.stock = self.stock - qty
        self.sold = self.sold + qty
        return f"{self.flavor} {qty}개 판매"

    def __str__(self):
        return f"{self.flavor}: 판매 {self.sold}개, 매출 {self.sold * self.price:,}원"
```

__init__: 재료 셋, 판매 0

sell: if로 막고 상태 바꾸기

__str__: 매출 글씨

Python

출력이 없는 셀이에요. 틀을 만들기만 했어요.

6. 실습

붕어빵 노점의 하루 ②: 팔고, 막고, 매출 찍기

[2]
```python
pat = Stall("팥", 1000, 20)
cream = Stall("슈크림", 1200, 10)
print(pat.sell(5))
print(cream.sell(12))
print(cream.sell(4))
print(pat)
print(cream)
```

```
팥 5개 판매
슈크림 재고가 모자라요
슈크림 4개 판매
팥: 판매 5개, 매출 5,000원
슈크림: 판매 4개, 매출 4,800원
```

일	팥 stock	슈크림 stock
찍기	20	10
pat.sell(5)	15	10
cream.sell(12)	15	10
cream.sell(4)	15	6

12개는 막혔어요

재고 10개보다 많아서 if에서 return했어요. 재고는 그대로예요.

배운 것 모두

__init__으로 찍기, 메서드로 상태 바꾸기, if와 return으로 막기, __str__로 보기 좋게 찍기

6. 실습

카페 메뉴판: 손님용 줄 맞춤과 개발자용 확인

[3]

```python
class Drink:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __str__(self):                                  # 손님용
        return f"{self.name:<7}{self.price:>6,}원"

    def __repr__(self):                                 # 개발자용
        return f"Drink('{self.name}', {self.price})"

menu = [Drink("latte", 5000), Drink("tea", 4000), Drink("coffee", 4500)]
for d in sorted(menu, key=lambda d: d.price):
    print(d)
print([d for d in menu if d.price < 5000])
```

Python

```
tea     4,000원
coffee  4,500원
latte   5,000원
[Drink('tea', 4000), Drink('coffee', 4500)]
```

메뉴판은 __str__

13의 칸 맞추기로 줄을 맞춰 찍어요.

확인은 __repr__

5000원 미만을 골라 담은 리스트는 주문표 모양으로 보여요.

정리

막히면 오류의 마지막 줄을 봐요

1	`<__main__.Bread object at 0x...>` __str__이나 __repr__이 없거나 밑줄이 하나예요	__repr__ 만들기
2	**리스트 안만 <... object ...>** __str__만 만들었어요	__repr__도 만들기
3	`TypeError: __str__ returned non-string (type NoneType)` 안에서 print했어요	return으로
4	`TypeError: __str__ returned non-string (type int)` 숫자를 돌려줬어요	str()이나 f-string
5	`TypeError: can only concatenate str (not "Bread") to str` 글자에 붕어빵을 바로 더했어요	`str(a), f"{a}"`
6	`TypeError: '<' not supported between instances` 붕어빵끼리 비교할 기준이 없어요	`key=lambda b: b.price`
7	**재고가 마이너스가 돼요** 메서드에서 값을 확인하지 않았어요	if와 return으로 막기

낯선 글자가 나오면 "지금 파이썬이 어느 메서드를 찾았을까"부터 생각해요.

한 장 요약

`def __str__(self):`

print, str, f-string

`return f"..."`

꼭 글자를 돌려주기

`def __repr__(self):`

셀, 리스트 안, 확인할 때

Bread('팥', 1000)

repr은 코드 모양으로

__repr__만

print도 빌려 써요

`str(a)`

글자로 바꿔 잇기

`key=lambda b: b.price`

붕어빵 정렬

`self.stock - qty`

메서드로 상태 바꾸기

__str__은 손님용 메뉴판, __repr__은 만든 사람용 주문표예요

다음 단계: 26에서 틀을 물려받아 슈크림 붕어빵 틀을 만들어요